Indexes in MongoDB Atlas

The Pets application with faster lookups

Gregory S. DeLozier, PhD

The Problem

Collection size	Lookup with no index
4 pets	Instant
50,000 pets	Reads every document
5,000,000 pets	Slow, and getting slower

The work grows with the data.

What an Index Is

Without an index	With an index
Examine every document	Look up the value in sorted order
Collection scan	Index scan
Work grows with collection size	Work grows much more slowly

Like the index at the back of a book.

Which Pets Queries Need Help

Query	Filters on	Indexed?
Get, update, delete by identifier	_id	Yes, automatic
List all pets or owners	Nothing	An index cannot help
Does this owner still have pets?	owner_id	No

The last row is the one we fix.

Install the MongoDB Shell

Where	Command
macOS	`brew install mongosh`
Windows	`winget install MongoDB.Shell`
Linux or Codespace	Follow the official page
Node.js installed	`npx mongosh`

Current as of writing. Check the official page.

Connect to Atlas

```
export MONGODB_URI='mongodb+srv://USER:PASSWORD@YOUR-CLUSTER.mongodb.net/'
mongosh "$MONGODB_URI" --apiVersion 1
```

```
use pets
show collections
```

Make Enough Data

```
const ownerId = db.owners.insertOne(
  {name: "Bulk Owner", city: "Kent"}).insertedId
const types = ["dog", "cat", "mouse", "fish", "bird"]
const docs = []
for (let i = 0; i < 50000; i++) {
  docs.push({name: "pet" + i, type: types[i % 5],
             age: i % 20, owner_id: ownerId})
}
db.pets.insertMany(docs)
```

Indexes pay off on large collections.

Reading a Query Plan

```
const lonelyId = db.owners.insertOne(
  {name: "No Pets", city: "Kent"}).insertedId
db.pets.find({owner_id: lonelyId})
  .explain("executionStats")
```

Look for	Meaning
COLLSCAN	Whole collection read
IXSCAN	Index used
totalDocsExamined	Documents read
nReturned	Documents returned

Before the Index

Measure	Expected value
Stage	COLLSCAN
Documents examined	Whole collection
Documents returned	0

Read everything to find nothing.

Create an Index

```
db.pets.createIndex({owner_id: 1},
                    {name: "pets_owner_id"})
db.pets.getIndexes()
```

Safe to repeat. An identical index is not created twice.

After the Index

Measure	Expected value
Stage	IXSCAN
Documents examined	0
Documents returned	0

Looked up the value. Read no pets.

Selectivity

Query	Matches	Index helps
A specific pet name	About 1 in 50,000	A great deal
type is dog	About 1 in 5	Much less

How much an index helps depends on how few documents the query matches.

Compound Indexes

```
db.pets.createIndex({owner_id: 1, type: 1},
                    {name: "pets_owner_id_type"})
```

Query on	Served by this index
owner_id and type	Yes
owner_id alone	Yes
type alone	Not efficiently

Unique Indexes

A unique index rejects a second document with the same value in the indexed field.

Pets and owners can share names, so we do not use one here.

What Indexes Cost

Operation	Effect of an added index
Read on the indexed field	Faster
Read on another field	No change
Insert, update, delete	Slower
Storage	More used

Index what your queries actually use.

Indexes at Startup

```
INDEXES = [
    ("pets", "owner_id", "pets_owner_id", {}),
    ("pets", "name", "pets_name", {}),
    ("pets", "type", "pets_type", {}),
    ("owners", "name", "owners_name", {}),
]
```

setup_database() calls ensure_indexes().

Plans from Python

```
python explain-queries.py
```

```
result = collection.find(query).explain()
```

Prints the stage, documents examined, and documents returned.

Takeaways

Index the fields your queries filter and sort on.

Use explain to confirm the index is used.

Selectivity decides how much an index helps.

Every index costs storage and write time.